

Relatório do Trabalho Prático

Disciplina: **Introdução a Sistemas Operacionais**

Tema: **Gerenciamento de Memória**

ARTHUR ROCHA COUTINHO

arthurrcouti@gmail.com

INTRODUÇÃO

Este relatório foi feito em função do trabalho prático da disciplina de Introdução a Sistemas Operacionais, da UFMS CPTL, e segue o modelo disponibilizado no enunciado. Este relatório tem por finalidade reportar sobre cada um dos algoritmos implementados, assim como também dos resultados e a análise dos resultados, além de indicar qual linguagem de programação e versões utilizadas com instruções para compilar e executar o programa. O programa foi implementado na linguagem de programação **JAVA**, que simula o gerenciamento de páginas em memória que foi visto durante a disciplina. Os algoritmos implementados foram o **FIFO**, **LRU**, **LFU** e **MFU**.

Nota do autor: A gravação de apresentação do trabalho não foi feita por incapacidade do hardware disponível de codificar o vídeo, até mesmo com todas as configurações no mínimo.

SEÇÃO 1 - COMPILAR E EXECUTAR

Para compilar o programa recomenda-se rodar o comando **“./build.sh”**, rodando o arquivo de build disposto no diretório, ao invés de compilar manualmente.

O programa foi desenvolvido com a versão **javac/openjdk 26.0.1**. Para mais informações consulte o Anexo 1.

A entrada do programa (sem interface gráfica) deve ser feita assim:

Shell

```
java -jar PageSimulator.jar [diretório_das_páginas]
[algoritmo_de_substituição_de_páginas]
[número_de_frames_de_memória] [quantidade_de_páginas_únicas]
[quantidades_de_páginas_requeridas]
```

Ex: *java -jar PageSimulator.jar /home/teste LRU 3 10 40*

Note que o script **“build.sh”** cria um diretório chamado **“diretorio_das_paginas”** no diretório de sua execução. Ele serve para ter o caminho passado como parâmetro no lugar

[diretório_das_páginas] no comando modelo. Porém o uso desse diretório é opcional para o correto funcionamento do programa.

SEÇÃO 2 – ALGORITMOS

Os algoritmos foram implementados seguindo o *Design Pattern Strategies*.

Vamos ver um pouco mais sobre os conceitos antes de ver como eles se comportam na prática

2.1 – FIFO (First-In, First-Out)

O algoritmo FIFO baseia-se em uma fila simples para gerenciar as páginas na memória.

- **Conceito:** A página que entrou primeiro na memória é a primeira a ser removida.

2.2 – LRU (Least Recently Used)

O LRU foca no histórico recente de acesso para prever o comportamento futuro do sistema.

- **Conceito:** Substitui a página que ficou mais tempo sem ser acessada.

2.3 – LFU (Least Frequently Used)

O LFU monitora a intensidade de uso das páginas por meio de um contador numérico.

- **Conceito:** Remove a página que acumulou o menor número de acessos até o momento.

2.4 – MFU (Most Frequently Used)

O MFU opera sob uma premissa inversa à do algoritmo LFU.

- **Conceito:** Substitui a página que possui o maior número de acessos registrados.

SEÇÃO 3 – RESULTADOS E ANÁLISES

Os testes foram realizados seguindo a metodologia especificada pelo enunciado do trabalho prático. Para cada algoritmo de substituição, foram realizadas 10 simulações, das quais calculou-se a média de falhas de página.

Metodologia de Análise:

- **Sequência de Requisições:** Tamanho fixo de 50 requisições.
- **Configurações de Memória:** Foram testados cenários de memória principal pequena (4 frames), média (10 frames) e grande (25 frames), visando uma simulação de 50 requisições.
- **Memória Virtual:** a memória virtual não foi especificada no enunciado, portanto, foi determinado de forma arbitrária 25 páginas como valor fixo para todas as simulações.
- **Simulações:** Foram realizadas 10 simulações para cada algoritmo (3 para memória pequena, 4 para média e 3 para grande).

Observação: Os tamanhos dos frames são estimativas arbitrárias e não buscam simular fielmente um sistema operacional real.

Geração de Dados:

Para a sequência de 50 páginas requisitadas, foi utilizada inteligência artificial para gerar 10 listas distintas para serem requisitadas para cada um dos algoritmos.

Prompt para geração de Dados:

None

APENAS gere uma lista de 10 listas de 50 inteiros cada, onde:

A lista dos inteiros deve obedecer os limites de 3 listas com inteiros de 0 a 24.A distribuição dos inteiros dentro da lista deve obedecer o Princípio da Localidade de Referência: Localidade Temporal, e, Localidade Espacial.O formato deve ser compatível para a entrada de um programa rodando em um terminal sem interface.

Data: 24/06/2026 - modelo Gemini 3.5 Flash

Saída do prompt:

None

```
11 12 13 14 11 12 13 15 14 13 2 3 4 5 2 3 4 6 5 4 18 19 20 21 18 19 20 22 21 20
11 12 13 14 11 12 13 15 14 13 2 3 4 5 2 3 4 6 5 4
5 6 7 8 5 6 7 9 8 7 15 16 17 18 15 16 17 19 18 17 0 1 2 3 0 1 2 4 3 2 5 6 7 8 5
6 7 9 8 7 15 16 17 18 15 16 17 19 18 17
20 21 22 23 20 21 22 24 23 22 8 9 10 11 8 9 10 12 11 10 14 15 16 17 14 15 16 18
17 16 20 21 22 23 20 21 22 24 23 22 8 9 10 11 8 9 10 12 11 10
12 13 14 15 12 13 14 16 15 14 3 4 5 6 3 4 5 7 6 5 21 22 23 24 21 22 23 24 23 22
12 13 14 15 12 13 14 16 15 14 3 4 5 6 3 4 5 7 6 5
1 2 3 4 1 2 3 5 4 3 10 11 12 13 10 11 12 14 13 12 17 18 19 20 17 18 19 21 20 19
1 2 3 4 1 2 3 5 4 3 10 11 12 13 10 11 12 14 13 12
16 17 18 19 16 17 18 20 19 18 7 8 9 10 7 8 9 11 10 9 0 1 2 3 0 1 2 4 3 2 16 17
18 19 16 17 18 20 19 18 7 8 9 10 7 8 9 11 10 9
9 10 11 12 9 10 11 13 12 11 22 23 24 23 22 23 24 22 23 24 4 5 6 7 4 5 6 8 7 6 9
10 11 12 9 10 11 13 12 11 22 23 24 23 22 23 24 22 23 24
13 14 15 16 13 14 15 17 16 15 1 2 3 4 1 2 3 5 4 3 19 20 21 22 19 20 21 23 22 21
13 14 15 16 13 14 15 17 16 15 1 2 3 4 1 2 3 5 4 3
6 7 8 9 6 7 8 10 9 8 18 19 20 21 18 19 20 22 21 20 11 12 13 14 11 12 13 15 14
13 6 7 8 9 6 7 8 10 9 8 18 19 20 21 18 19 20 22 21 20
2 3 4 5 2 3 4 6 5 4 14 15 16 17 14 15 16 18 17 16 21 22 23 24 21 22 23 24 23 22
2 3 4 5 2 3 4 6 5 4 14 15 16 17 14 15 16 18 17 16
```

3.1 – ANÁLISE FIFO (First-In, First-Out)

Média de falhas: $(21 + 21 + 21 + 20 + 21 + 21 + 17 + 21 + 21 + 20)/10 = 20.4$

- **Vantagem:** Muito simples de entender e implementar no sistema operacional.
- **Desvantagem:** Pode remover páginas que ainda são frequentemente utilizadas pelo sistema.

3.2 – ANÁLISE LRU (Least Recently Used)

Média de falhas: $(26 + 26 + 26 + 24 + 26 + 26 + 20 + 26 + 26 + 24)/10 = 25$

- **Vantagem:** Apresenta excelente desempenho prático ao se basear no princípio da localidade temporal.
- **Desvantagem:** Exige suporte de hardware caro ou alto processamento para rastrear o tempo de cada acesso.

3.3 – ANÁLISE LFU (Least Frequently Used)

Média de falhas: $(26 + 26 + 26 + 24 + 26 + 26 + 20 + 26 + 26 + 24)/10 = 25$

- **Vantagem:** Protege páginas que são usadas repetidamente em loops intensos do sistema.
- **Desvantagem:** Páginas muito usadas no início do programa acumulam contagens altas e podem nunca ser removidas, mesmo que fiquem obsoletas.

3.4 – ANÁLISE MFU (Most Frequently Used)

Média de falhas: $(21 + 21 + 21 + 20 + 21 + 21 + 17 + 21 + 21 + 20)/10 = 20.4$

- **Vantagem:** Teoria baseada na ideia de que páginas com baixa contagem foram trazidas recentemente e ainda serão usadas.
- **Desvantagem:** Raras vezes é utilizado na prática devido ao seu desempenho inferior na maioria dos cenários reais.

3.5 – ANÁLISE FINAL

A análise dos resultados revelou um cenário divergente da literatura clássica de Sistemas Operacionais. Embora algoritmos como LRU e LFU sejam teoricamente superiores por explorarem melhor a localidade de referência, os dados desta simulação apontaram um desempenho superior para FIFO e MFU.

Essa inversão de expectativas pode ser atribuída à natureza da carga de trabalho gerada sinteticamente e ao volume reduzido da amostra (50 requisições). Em sequências com padrões de acesso cíclicos ou de alta repetição, o FIFO pode apresentar métricas competitivas devido à sua baixa sobrecarga. Por outro lado, a dependência do LRU e do LFU em históricos de longo prazo pode ter sido subutilizada no volume de dados restrito utilizado.

Comparativo de Taxa de Falhas:

- Grupo FIFO e MFU: Média de 20.4 falhas (~40,8% de taxa de falha).
- Grupo LRU e LFU: Média de 25.0 falhas (~50,0% de taxa de falha).

Estes dados demonstram que, em ambientes controlados e com cargas de trabalho específicas, a simplicidade de implementação (como no FIFO) pode gerar resultados eficazes, ainda que tais algoritmos não reflitam a robustez exigida em sistemas operacionais de larga escala.

ANEXOS

Anexo 1 – Versões e Ambiente de execução

```
Simulador-de-Gerenciamento-de-Memoria-Principal-e-Virtual on  dev via  v26.0.1 took 14m55s
● > javac -version
javac 26.0.1

Simulador-de-Gerenciamento-de-Memoria-Principal-e-Virtual on  dev via  v26.0.1
● > java -version
openjdk version "26.0.1" 2026-04-21
OpenJDK Runtime Environment (Red_Hat-26.0.1.0.8-1) (build 26.0.1+8)
OpenJDK 64-Bit Server VM (Red_Hat-26.0.1.0.8-1) (build 26.0.1+8, mixed mode, sharing)

Simulador-de-Gerenciamento-de-Memoria-Principal-e-Virtual on  dev via  v26.0.1
● >
```

Anexo 2 - Tabela de resultados

Algoritmo	Média de Falhas	Taxa de Falha
FIFO	20,4	~40,8%
LRU	25,0	~50,0%
LFU	25,0	~50,0%
MFU	20,4	~40,8%

CONCLUSÃO

Conclui-se, a partir da realização deste trabalho prático, que a implementação dos algoritmos de substituição de páginas (FIFO, LRU, LFU e MFU) foi fundamental para a compreensão dos mecanismos de gerenciamento de memória. Embora os resultados obtidos na simulação tenham divergido da teoria clássica — apresentando um comportamento atípico onde FIFO e MFU performaram de forma superior a LRU e LFU —, essa divergência serviu como um estudo de caso valioso sobre a importância da carga de trabalho da máquina.

Ficou evidente que a qualidade, a natureza dos dados de entrada e o tamanho da amostra impactam diretamente a eficácia de cada estratégia. A discrepância observada não invalida o aprendizado, mas destaca que algoritmos de substituição não operam em um vácuo e são

altamente dependentes dos padrões de acesso de cada aplicação específica. Em última análise, o trabalho cumpriu seu objetivo pedagógico, permitindo a transposição da teoria acadêmica para uma aplicação prática funcional em Java, fornecendo uma base sólida para futuras investigações sobre sistemas operacionais.